

Model Quality vs. the LARA Harness and Architecture: Which Contributes More, and When?

LARA Project — Experimental Report

Compiled 2026-09-05 · all scores recomputed from stored run outputs

Abstract. A capable agent on the AppWorld benchmark draws on two separable sources of strength: the quality of the underlying language model, and the harness and architecture wrapped around it — here, LARA, which separates an *Explorer* planning stage from an *Executor*. We ask which of the two contributes more, and find that the answer depends entirely on task difficulty. Filling a 2×2 grid of two models (gpt-4.1-mini, claude-opus-4-7) against two agents (the benchmark's minimal reference loop, and LARA) across two splits, we measure each contribution with the other held fixed. On easy tasks a strong model largely stands on its own: with minimal architecture it already scores 87.7 TGC, and LARA adds only 10.5 points. On the hardest tasks the same strong model collapses to 10.3 without the architecture, and LARA lifts it to 84.1 — a 73.8-point rescue. The model buys the baseline capability; the architecture buys survival as tasks lengthen. LARA with claude-opus-4-7 reaches 85.6 TGC on test_challenge, first on the public leaderboard as of 2026-09-06, and neither the model nor the architecture alone comes close to that score.

Keywords: LLM agents, task planning, tool use, AppWorld, model scaling, agent architecture.

1. Introduction

When an LLM agent succeeds at a multi-app task, credit is easy to misassign. A stronger base model and a better-engineered harness both raise scores, and a single leaderboard number cannot say which did the work. Yet the distinction is practical: it decides whether the next gain comes from paying for a larger model or from investing in the scaffold around it.

This report separates the two contributions on the AppWorld benchmark. On one axis is *model quality* — here gpt-4.1-mini versus claude-opus-4-7. On the other is the *harness and architecture* — the benchmark's own minimal ReAct reference agent, versus LARA, which restructures the problem into an Explorer that plans against the real API surface and an Executor that carries the plan out with per-app knowledge and a cross-app ledger. By filling every cell of the two-model × two-agent grid on both the test_normal and test_challenge splits, we can hold one axis fixed and read off the other, and — crucially — watch how each contribution changes as tasks get harder.

The finding is that the two are not competing for a fixed pool of credit; they dominate in different regimes. Model quality is what carries easy tasks: a strong model needs little help there. Architecture is what carries hard tasks: without it, even a strong model runs out of steps before it can act. The best result requires both, and the crossover is governed by difficulty.

2. The crossover, in one table

LARA was submitted to the public AppWorld leaderboard on 2026-09-03. On test_challenge (417 tasks) it reaches **85.6 TGC / 77.0 SGC** with claude-opus-4-7, taking **first place as of 2026-09-06**, 12.2 TGC points clear of the second entry (73.4); on test_normal (168 tasks) it reaches 88.7 / 82.1. Both bundles were independently re-scored by CI and reproduced exactly.

The whole argument can be read from four numbers. Table 1 takes the strong model alone and asks what the architecture adds at the two extremes of difficulty. On easy tasks it adds almost nothing; on hard tasks it is the difference between collapse and success.

split	difficulty	minimal harness	+ LARA	Δ architecture
test_normal	d1 (easy)	87.7	98.2	+10.5
test_normal	d3 (hard)	27.0	79.4	+52.4
test_challenge	d1 (easy)	75.0	91.7	+16.7
test_challenge	d3 (hard)	10.3	84.1	+73.8

Table 1. The crossover, isolated: claude-opus-4-7 at the easiest and hardest tiers. On easy tasks the architecture is nearly redundant (Δ +10.5, +16.7); on hard tasks the same strong model depends on it entirely (Δ +52.4, +73.8).

The rest of this section unpacks those four numbers into the full grid, broken out by difficulty tier (Tables 2a–2c) and visualised in Figure 1. Throughout, Δ arch is the gain from replacing the minimal harness with LARA at a fixed model, i.e. the architecture’s isolated contribution.

3. The full grid, by difficulty

Each table below is one difficulty tier and holds all four model×agent cells for both splits; the claude-opus-4-7 rows (shaded) carry the cleanest signal because the strong model can actually use what the architecture provides. Reading the Δ arch column down the three tables shows it expanding as tasks harden.

split	model	minimal	LARA	Δ arch
test_normal	gpt-4.1-mini	47.4	91.2	+43.8
test_normal	claude-opus-4-7	87.7	98.2	+10.5
test_challenge	gpt-4.1-mini	36.1	72.2	+36.1
test_challenge	claude-opus-4-7	75.0	91.7	+16.7

Table 2a. Difficulty 1 (easy). With a strong model, the minimal harness already scores high (87.7, 75.0) and the architecture adds little (+10.5, +16.7).

split	model	minimal	LARA	Δ arch
test_normal	gpt-4.1-mini	18.8	52.1	+33.3
test_normal	claude-opus-4-7	52.1	89.6	+37.5
test_challenge	gpt-4.1-mini	8.0	36.0	+28.0
test_challenge	claude-opus-4-7	26.7	84.7	+58.0

Table 2b. Difficulty 2 (medium). The strong model has begun to fall behind without the architecture; Δ grows to +37.5 and +58.0.

split	model	minimal	LARA	Δ arch
test_normal	gpt-4.1-mini	4.8	42.9	+38.1
test_normal	claude-opus-4-7	27.0	79.4	+52.4
test_challenge	gpt-4.1-mini	1.5	26.2	+24.7
test_challenge	claude-opus-4-7	10.3	84.1	+73.8

Table 2c. Difficulty 3 (hard). The strong model collapses without the architecture (27.0, 10.3); LARA rescues it by +52.4 and +73.8 — the largest gaps measured.

The pattern in the shaded rows is the crossover stated numerically: for the strong model, Δ arch climbs from +10.5 / +16.7 at d1, through +37.5 / +58.0 at d2, to +52.4 / +73.8 at d3. The weaker model's Δ does not climb as cleanly — on the hardest tier it cannot fully exploit the architecture (a floor effect visible in Figure 1) — which is itself the point: the largest architectural payoff appears where a *capable* model meets a *hard* task, not from either ingredient alone.

The scaffold gap widens as tasks get harder

Minimal ReAct agent vs. LARA on AppWorld, by difficulty — shaded band = the gain LARA adds

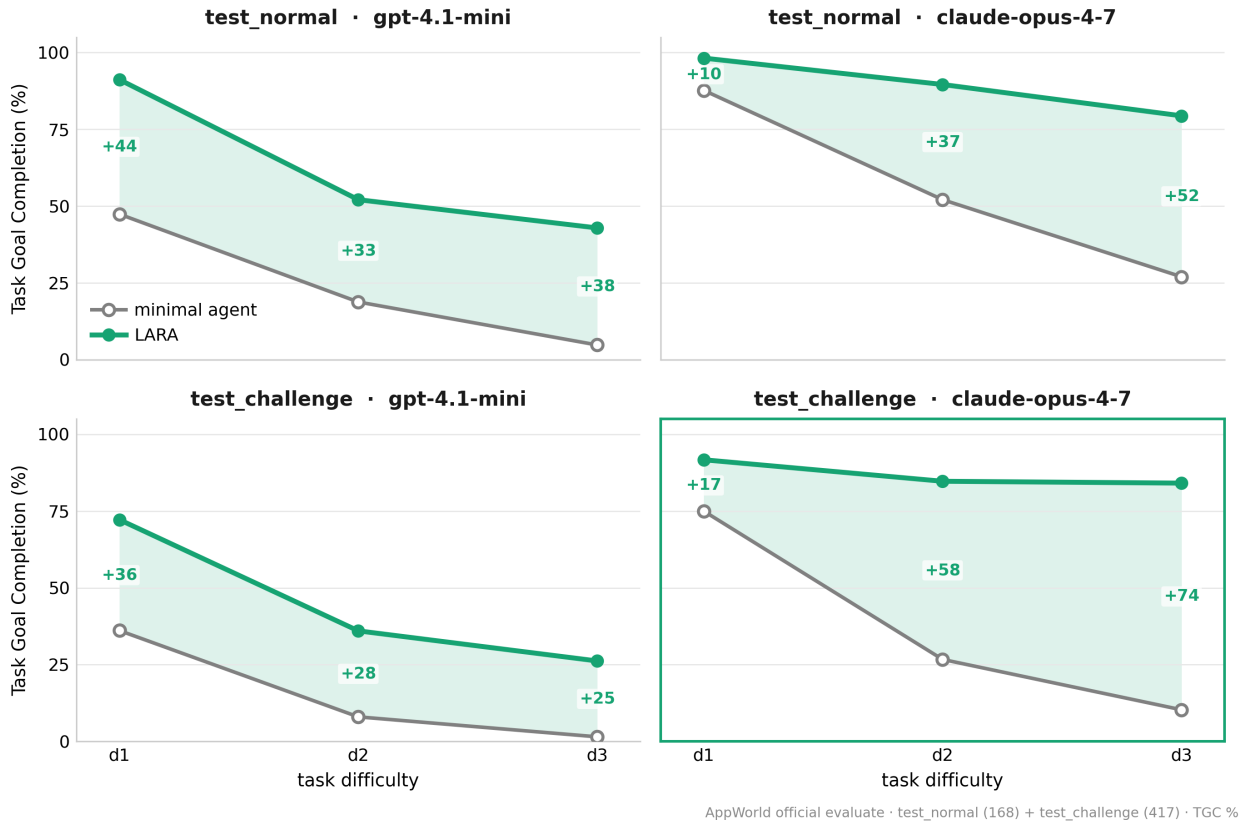


Figure 1. Reference agent (grey) versus LARA (green) on AppWorld, broken out by task difficulty; the shaded band is the Δ arch of Tables 2a-2c, with the model held fixed. In both claude-opus-4-7 panels the band is narrow at d1 and widens monotonically, reaching +74 points at difficulty-3 on test_challenge (highlighted). The gpt-4.1-mini panels show the floor — the weaker model cannot fully use the architecture on the hardest tier — so the largest architectural payoff appears exactly where a capable model meets a hard task.

Easy tasks — the model carries it.

Take the strong model on the easiest tier (Table 2a). With only the minimal reference harness it already scores 87.7 TGC on test_normal and 75.0 on test_challenge; adding the

full LARA architecture lifts these by just +10.5 and +16.7. The model is close to self-sufficient here — the plan and per-app knowledge have little left to contribute because a short task rarely exhausts the step budget or demands cross-app memory. If every task looked like this, a strong model on a bare loop would be a defensible configuration.

Hard tasks — the model needs the architecture.

The same strong model tells the opposite story on the hardest tier (Table 2c). The minimal harness drops it to 27.0 on `test_normal` and 10.3 on `test_challenge` — near-total collapse for a top model. LARA rescues the same model to 79.4 and 84.1, gains of +52.4 and **+73.8** points. Here model quality alone is not enough: the task is long enough that an unaided agent spends its budget rediscovering APIs and never submits, no matter how strong the model. The architecture is what converts a capable model into a successful one. The aggregate reflects this directly: the reference agent with the strong model drops 54.8 → 27.3 between splits (a 50% loss), while LARA on the same model drops only 88.7 → 85.6 (3.5%).

Failure mechanism. Across the reference-agent failures, 71% on `test_challenge` (267/376) and 70% on `test_normal` (91/129) are `no_submit`: the agent exhausts its 16-step budget without calling `complete_task()`, spending it discovering which APIs exist and hallucinating ones that do not (e.g. `set_access_token`, `show_account_usernames`, `list_transactions`). This is why a better model alone cannot save the hard tasks — the bottleneck is budget spent on discovery, not reasoning quality — and why moving discovery into LARA's separate Explorer stage, which never touches the Executor's step budget, is what closes the gap.

4. Which would you rather have? Architecture on a cheap model vs. a strong model alone

The crossover has a sharp practical corollary. Suppose the choice were forced: spend the budget on a stronger model but run it on the bare reference loop, or keep a cheap model but wrap it in LARA. Table 1 and the difficulty tables let us answer directly, because both configurations were measured — LARA on `gpt-4.1-mini` (cheap model, good harness) against the minimal agent on `claude-opus-4-7` (strong model, bare harness).

The cheap-model-with-architecture configuration wins almost everywhere. In aggregate it leads by +7.1 TGC on `test_normal` (61.9 vs 54.8) and +10.3 on `test_challenge` (37.6 vs 27.3). The only cell the strong model wins is the very easiest — difficulty-1 on `test_challenge`, by 2.8 points — and from there the ordering reverses: as tasks harden, LARA-on-mini pulls ahead by +9 at d2 and +16 at d3 (Figure 2). On `test_normal` the cheap configuration is never behind, tying at d2 and leading by +16 at the hardest tier.

When cheap-model-plus-architecture overtakes strong-model-alone

Bold number = margin of LARA+mini over minimal+opus at each difficulty tier

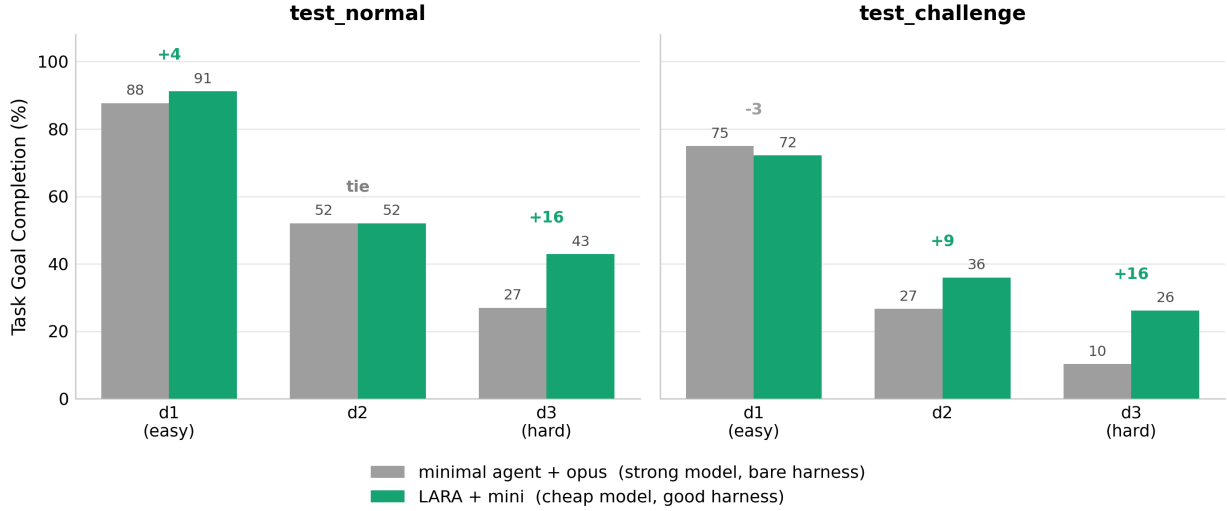


Figure 2. LARA on the cheaper gpt-4.1-mini (green) versus the minimal agent on the stronger claude-opus-4-7 (grey), by difficulty tier. Bold numbers give the margin of the cheap-model-with-architecture configuration over the strong-model-alone one. The strong model leads only at the easiest tier of test_challenge (-3); at every harder tier the cheap-model-with-architecture configuration is ahead, by up to +16 points on both splits.

The reading is not that the model does not matter — Section 3 showed it is worth up to 48 points — but that **a good harness on a weak model beats a bare harness on a strong one, and the advantage grows with difficulty**. Only on the shortest, easiest tasks, where the harness has little to do, does raw model quality win outright. Everywhere the benchmark is actually hard, the architecture is the better place to spend.

5. Method

Leaderboard runs used the held-out test_normal (168 tasks) and test_challenge (417 tasks) splits. Accuracy is reported as TGC and SGC from AppWorld's official evaluate; d1-d3 are the official per-difficulty TGC figures. Each task received one Executor attempt, with Reviewer and Supervisor retry disabled and a 16-step ReAct budget. Every contribution in Sections 2-4 is read with the other axis held fixed, so each model comparison shares an agent and each architecture comparison shares a model. All figures are recomputed from stored run outputs, and both leaderboard bundles were independently re-scored by CI and reproduced exactly.

6. Limitations

Several constraints bound how far the conclusions above should be pushed.

The two axes are not on a common scale. We measure the model contribution and the architecture contribution separately, but the size of the step taken along each axis is arbitrary and unmatched. The "model" step is a whole generation-and-price jump (gpt-4.1-mini to claude-opus-4-7); the "architecture" step is minimal-loop to full LARA. There is no shared unit that would let us say one axis matters more than the other in general — only within these particular endpoints. A closer pair of models could have made the model difference look negligible beside the architecture, and a weaker harness could have reversed that. Every "which contributes more" statement here is relative to the specific steps tested, not an intrinsic ranking.

Only two models, both at the extremes. The grid has a cheap-weak model and an expensive-strong one, with nothing in between. We therefore cannot tell whether the crossover in Section 4 is sharp or gradual, or whether a mid-tier model would sit closer to one story or the other. The difficulty at which architecture overtakes raw model quality is located only coarsely.

One run per cell, and hosted-model drift. Each cell is a single run; there are no within-condition repeats to estimate variance. A separate measurement in this project found the same code scoring about 11 points higher two weeks later on a hosted model, so cross-run differences of a few points are not safely outside the noise. The small margins in particular — the +4 and −3 at the easy tiers, and the ties — should be read as "close," not as established orderings; only the large, consistent gaps (the double-digit leads on hard tasks) are robust to this.

The "cheap vs. strong" comparison is on accuracy, not cost. Section 4 shows LARA on the cheaper model beats the minimal agent on the stronger one *in TGC*, but it does not net out the harness's own overhead: LARA consumes substantially more tokens per task than the bare reference loop. The claim is therefore that architecture-on-a-cheap-model wins on task success, not that it is necessarily cheaper per task or better on a combined cost-accuracy basis. A fair budget-matched comparison would need to price the model saving against the harness's token cost, which we did not do here.

7. Discussion and conclusion

Model quality and the LARA harness are not rivals for a fixed pool of credit; they dominate in different regimes, and difficulty is the dial that selects between them. On easy tasks the model is decisive: a strong model on a minimal harness is already near its ceiling, and the architecture adds little. On hard tasks the architecture is decisive: the same strong model collapses without it, and LARA restores it by up to 74 points. The practical reading follows directly — **on relatively easy workloads a good model succeeds even with very basic architecture, but on complex, multi-app tasks that model must be backed by a strong harness to succeed at all.**

This also explains why the submitted result needs both. The model is worth up to 48 TGC points and the architecture up to 58, and neither alone reaches 85.6: the strong model without the harness scores 27.3 on `test_challenge`, and the harness without a strong model (LARA on `gpt-4.1-mini`) scores 37.6. The leaderboard entry is the product of pairing a capable model with an architecture that keeps it from running out of steps — and the harder the benchmark grows, the more that pairing, rather than either half of it, is what matters.

Reproducibility and compliance. All figures are recomputed from stored run outputs; both leaderboard bundles were independently re-scored by CI and reproduced exactly. Authentication was moved into the prompt (permitted by the rules); no `apis.*` calls remain in live code (AST-verified 2026-09-02); and held-out splits were scored but never analysed — no evaluation report was opened for any test run.